

INSTITUTO PROFESIONAL
SAN SEBASTIAN

Digitalización inicial de solicitudes de venta

Hielos del Maipo

INFORME TÉCNICO · EVALUACIÓN SUMATIVA U1

Asignatura: Desarrollo Backend (IF201IINF)

Estudiante: Franco Ramírez

Docente: Raul Bilbao Barrenechea

Fecha: 1 noviembre de 2026

Repositorio: github.com/francoramirezcontacto-eng/DesarrolloBackendEV1

1. Contexto y objetivo del trabajo

Hielos del Maipo vende hielo al por mayor y menor. En la ficha entregada por la empresa se mencionan problemas para ordenar las ventas, las rutas y algunos registros que hoy se llevan en planillas y cuadernos. Para esta primera unidad trabajé con una parte pequeña del problema: registrar solicitudes de pedido de forma ordenada y poder consultarlas después.

El objetivo fue construir una API REST en PHP con Laravel 12 y una base de datos SQLite que guarde realmente los datos. La aplicación se probó de forma local con Postman y con pruebas automatizadas. No se desarrolló una tienda online ni se conectó todavía con la página de Frontend, porque eso no es parte del alcance solicitado para esta evaluación.

Los campos y estados que se usan son ejemplos para aprender y comprobar el funcionamiento. Todavía habría que confirmarlos con Hielos del Maipo antes de usar el sistema con clientes reales.

2. Arquitectura y rol del backend

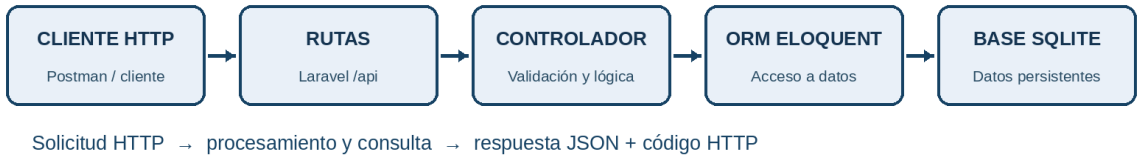


Figura 1. Arquitectura utilizada para procesar las solicitudes.

Cuando Postman envía una solicitud HTTP, Laravel busca la ruta correspondiente y ejecuta el controlador. Si llegan datos para crear o modificar un pedido, la clase FormRequest revisa que sean válidos. Luego, Eloquent hace la operación en SQLite y el servidor entrega una respuesta JSON junto con su código HTTP.

Componente	Función en el proyecto
Cliente HTTP (Postman)	Permite enviar y comprobar solicitudes GET, POST, PUT, PATCH y DELETE.
Laravel y rutas	Reciben las peticiones y las envían al controlador correspondiente.
Controlador / FormRequest	Coordinan las operaciones y revisan los datos recibidos.
Eloquent ORM	Permite trabajar con registros sin escribir consultas SQL concatenadas.
SQLite	Almacena los pedidos localmente y mantiene la información después del reinicio.

Elegí una estructura separada por responsabilidades para que el código sea más fácil de entender y mantener. También ayuda a encontrar errores: una validación se corrige en su clase y no hay que modificar toda la API. El manejo de errores y la paginación aportan robustez y una base para ampliar el sistema en futuras unidades.

3. Base de datos y operaciones disponibles

El sistema trabaja con una sola entidad llamada SolicitudPedido y con la tabla solicitud_pedidos. La estructura se creó con una migración de Laravel. El archivo database/database.sqlite se mantiene localmente y los cambios se hacen mediante el modelo Eloquent.

Campo	Tipo	Uso
id	Identificador	Distingue una solicitud de otra.
cliente_referencia	Texto, máximo 120	Nombre o referencia ficticia del cliente.
cantidad	Entero positivo	Cantidad solicitada en unidades de prueba.
fecha_requerida	Fecha	Fecha registrada con formato AAAA-MM-DD.
estado	Texto	pendiente, en_revision o finalizada.
created_at / updated_at	Fecha y hora	Control de creación y última modificación.

Rutas de la API

Método	Ruta	Respuesta comprobada o prevista
GET	/api/solicitudes	200: lista paginada.
GET	/api/solicitudes/{id}	200: detalle; 404 si no existe (en pruebas automáticas).
POST	/api/solicitudes	201: crea; 422 si los datos son incorrectos.
PUT	/api/solicitudes/{id}	200: actualiza todos los campos.
PATCH	/api/solicitudes/{id}	200: actualiza solo los campos enviados (en pruebas).
DELETE	/api/solicitudes/{id}	204: elimina y no devuelve contenido.

Ejemplo de solicitud POST

```
{ "cliente_referencia": "Cliente de prueba 01", "cantidad": 8, "fecha_requerida": "2026-11-15", "estado": "pendiente" }
```

En esta prueba, el servidor devolvió el código 201 y generó el identificador 1. Después se consultó el mismo registro y se actualizó la cantidad de 8 a 12. Todo se hizo usando datos ficticios.

4. Validaciones y criterios de calidad

La lógica de negocio se mantuvo simple: la referencia del cliente es obligatoria, la cantidad debe ser un entero mayor que cero, la fecha debe respetar el formato solicitado y el estado debe pertenecer a una lista de tres opciones. Estas reglas son suficientes para el ejercicio, pero no representan procesos comerciales confirmados por la empresa.

En Postman probé una cantidad negativa (-5) y un estado inexistente (desconocido). La API respondió 422 y señaló por separado los dos errores: la cantidad debe ser mayor que cero y el estado debe ser pendiente, en_revision o finalizada.

Lista de revisión aplicada al prototipo

Criterio observable	Resultado real
Código ordenado	Rutas, controlador, modelo y validaciones separados en archivos.
Respuestas HTTP coherentes	Se observaron 201, 200, 204 y 422; las pruebas también comprobaron 404.
Errores comprensibles	Postman muestra mensajes claros para cantidad y estado incorrectos.
Datos persistentes	El pedido con cantidad 12 siguió apareciendo al reiniciar Laravel.
Consultas acotadas	Listado paginado con 10 elementos por defecto y máximo de 50.
Verificación automática	PHPUnit terminó con 6 pruebas aprobadas y 30 aserciones.

Problema encontrado y mejora aplicada

Durante las primeras pruebas apareció un error en la actualización parcial PATCH. Una regla se había escrito como un solo texto (sometimes|required) dentro del arreglo de validaciones, por lo que Laravel intentaba interpretarla como un método que no existía. Se corrigió separando las reglas en elementos individuales y repitiendo las pruebas.

Antes de la corrección el resultado era 5 pruebas aprobadas y 1 fallida. Después quedaron 6 pruebas aprobadas y 30 aserciones. Esta mejora concreta permitió que

PUT siguiera exigiendo todos los campos, mientras PATCH aceptara modificar únicamente los datos enviados.

5. Pruebas realizadas y resultados

Las solicitudes HTTP se enviaron desde Postman al servidor local http://127.0.0.1:8000. Además, se ejecutó PHPUnit con el comando php artisan test --filter=SolicitudPedidoApiTest. Estos son los resultados observados en las capturas y en PowerShell:

Prueba	Resultado	Qué se comprobó
POST /api/solicitudes	201 Created	Registro ID 1 creado con cantidad 8.
GET /api/solicitudes	200 OK	La lista devolvió el registro creado.
PUT /api/solicitudes/1	200 OK	La cantidad cambió de 8 a 12.
GET /api/solicitudes/1 tras reinicio	200 OK	Se mantuvo la cantidad 12.
DELETE /api/solicitudes/1	204 No Content	El servidor confirmó la eliminación.
POST con cantidad -5 y estado inválido	422 Unprocessable Content	La API rechazó ambos valores con mensajes de error.
PHPUnit	6 passed, 30 assertions	Pruebas de CRUD, PATCH, validaciones, 404 y paginación.

Para comprobar la persistencia detuve el servidor con Ctrl + C, lo inicié nuevamente con php artisan serve y consulté el mismo ID. La respuesta devolvió el pedido con la cantidad 12. Esto indica que la información estaba almacenada en SQLite y no solo en la memoria del servidor.

6. Rendimiento y escalabilidad

El problema que se quiso mejorar fue la entrega de demasiados registros en una sola consulta. Si una API devuelve todos los pedidos de una vez, el volumen de datos puede crecer sin necesidad. Por eso implementé paginación: se entregan 10 resultados por defecto y el máximo permitido es 50 por petición.

Medición local	Sin paginación	Con paginación
Registros devueltos	60	10
Tiempo observado	12,47 ms	0,87 ms
Páginas disponibles	—	6

Con el seeder se crearon 60 registros ficticios. En esta ejecución la consulta paginada trajo menos datos y terminó más rápido. Los milisegundos dependen del equipo y de las condiciones locales; por eso no corresponden a una prueba de carga formal. Si el sistema creciera, sería útil revisar índices, monitoreo y consultas antes de pensar en una arquitectura más grande.

7. Seguridad y límites de esta primera versión

Para manejar la base de datos se utilizó Eloquent ORM, que permite realizar las operaciones con parámetros en lugar de construir consultas SQL juntando texto ingresado por el usuario. La configuración de SQLite se controla desde el archivo .env, que no se incluye en GitHub junto con la base local, las dependencias instaladas o las claves del proyecto.

También es importante distinguir tres conceptos: autenticación es comprobar quién ingresa, autorización es definir qué puede hacer y TLS/HTTPS es proteger la comunicación. En esta unidad no se implementó un sistema de usuarios ni HTTPS de producción. Por ese motivo, la API se debe usar solamente en un entorno de desarrollo local y con datos ficticios.

8. Lo que queda pendiente para la empresa

Antes de ampliar el proyecto sería necesario confirmar cómo identifican a sus clientes, qué unidades de hielo manejan, qué estados utilizan realmente, quién modifica los pedidos y cómo se relacionan con producción y reparto. Esos puntos no se inventaron ni se incorporaron como funciones, ya que requieren conversación y validación con Hielos del Maipo.

En general, esta primera versión cumple la finalidad de tener un registro simple de solicitudes, que se puede crear, consultar, modificar y eliminar. El resultado más importante fue lograr un backend que guarda los datos de verdad, responde con códigos HTTP adecuados y permite comprobar su funcionamiento.

9. Declaración del uso de inteligencia artificial

Utilicé ChatGPT como herramienta de apoyo para ordenar las instrucciones y la rúbrica, plantear una estructura inicial del backend y del informe, y revisar código cuando aparecieron errores. La instalación de Laravel, la ejecución de las peticiones en Postman, la revisión de las respuestas, la corrección del problema con PATCH y las pruebas finales se realizaron en mi entorno de desarrollo. También revisé el contenido para entender qué hacía cada parte y dejar registrados los resultados reales, no respuestas inventadas.

Fuentes y material utilizado

Material de la asignatura (Instituto Profesional San Sebastián): Evaluación Sumativa U1, Rúbrica U1, Ficha VcM de Hielos del Maipo, Cuaderno de Ejercitación y sesión práctica S3.

Documentación oficial de Laravel 12: https://laravel.com/docs/12.x/.

Anexo A. Evidencias de funcionamiento

Las imágenes siguientes son recortes de las capturas reales tomadas durante las pruebas. En el repositorio se conservan otras evidencias de los métodos HTTP y del código. Los recortes muestran la solicitud y su resultado sin modificar los datos.

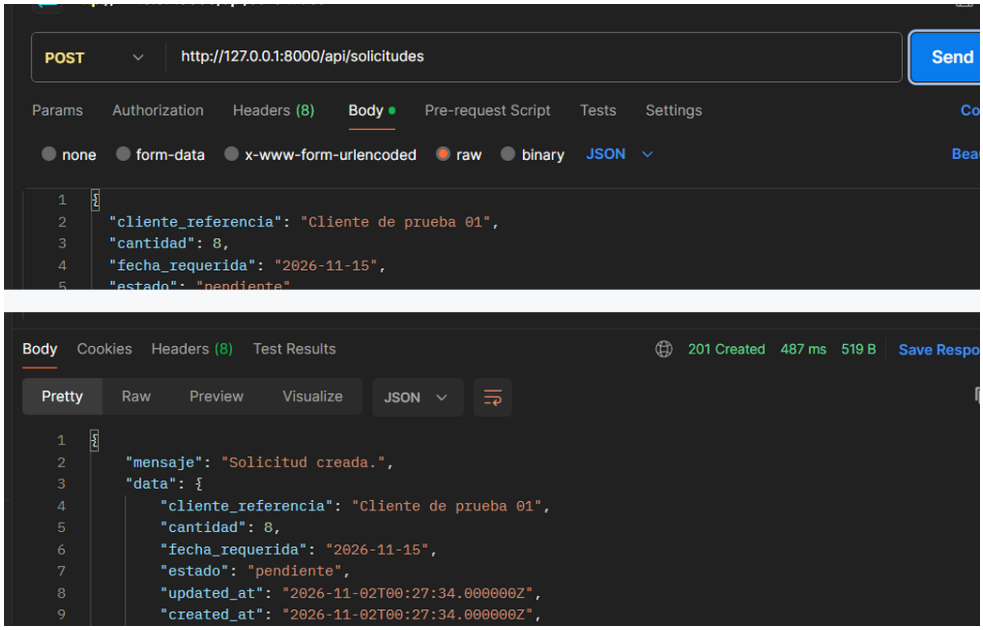


Figura 2. POST: creación de una solicitud de prueba con respuesta HTTP 201.

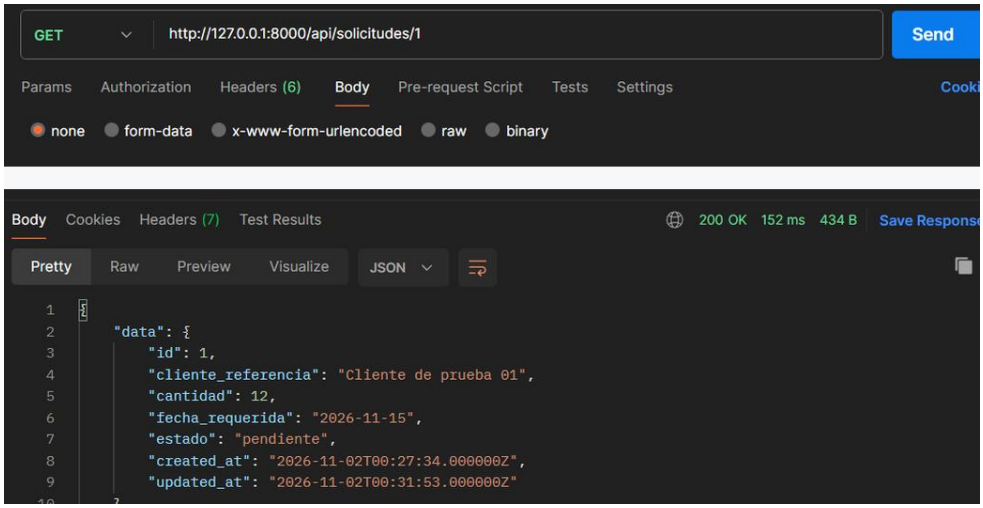


Figura 3. GET del ID 1 tras reiniciar el servidor: respuesta 200 y cantidad 12.

Anexo B. Validaciones y pruebas automáticas

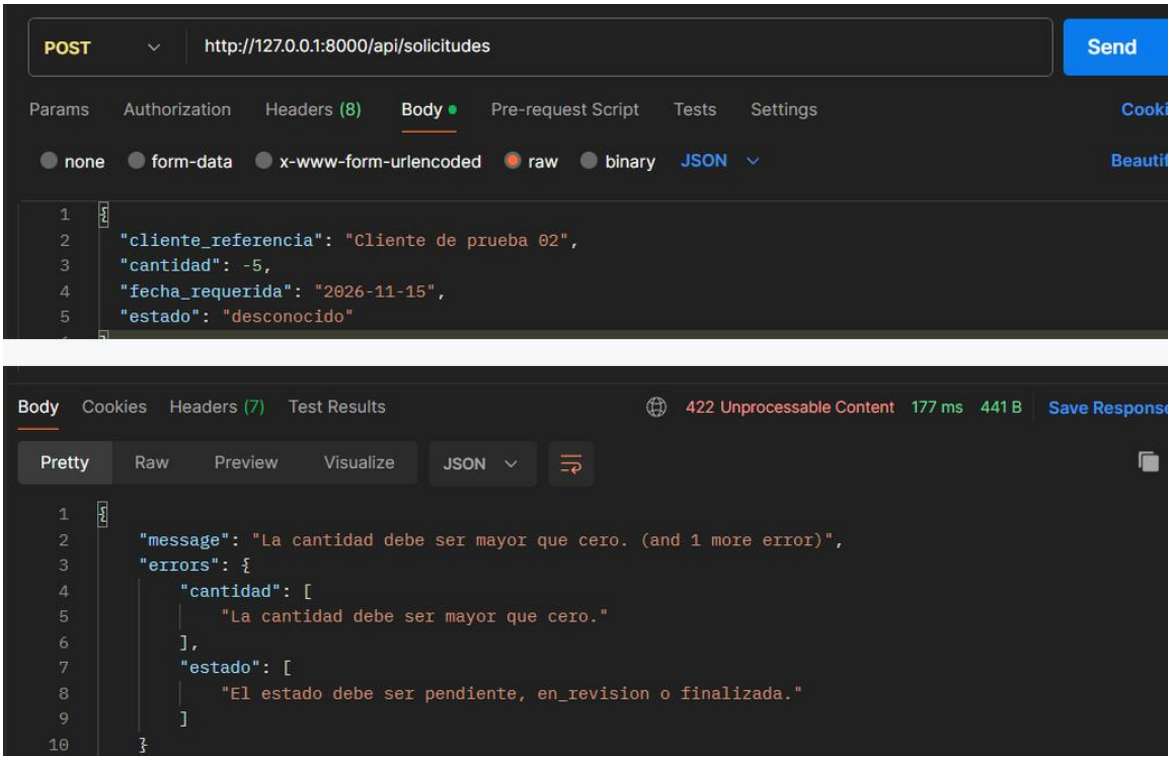


Figura 4. Validación de cantidad -5 y estado desconocido: HTTP 422.

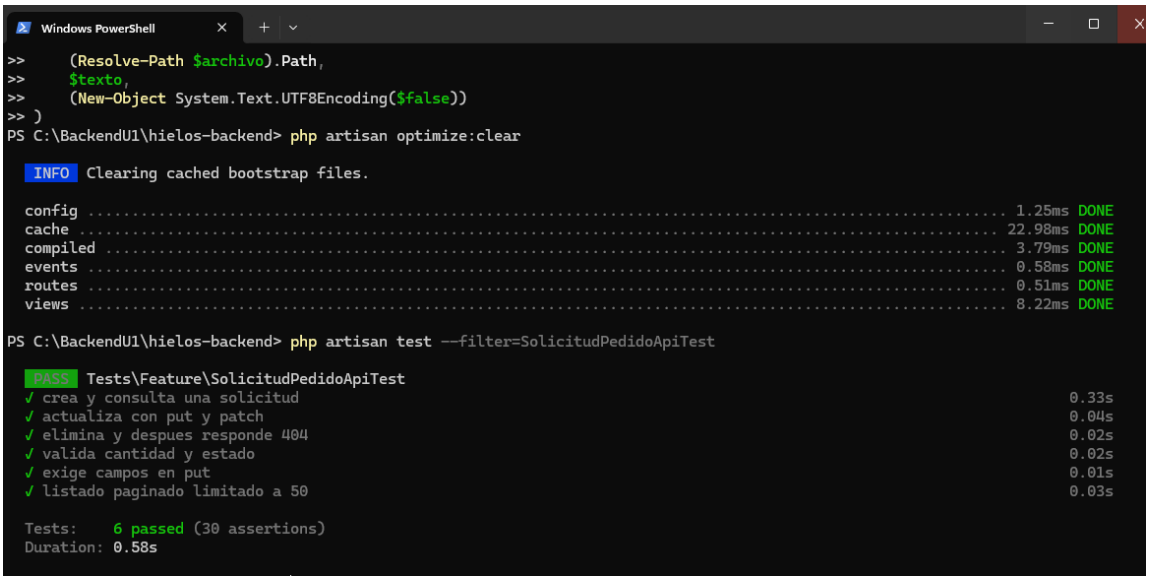


Figura 5. PHPUnit: 6 pruebas aprobadas y 30 aserciones.

Anexo C. Rendimiento y registro de evidencias

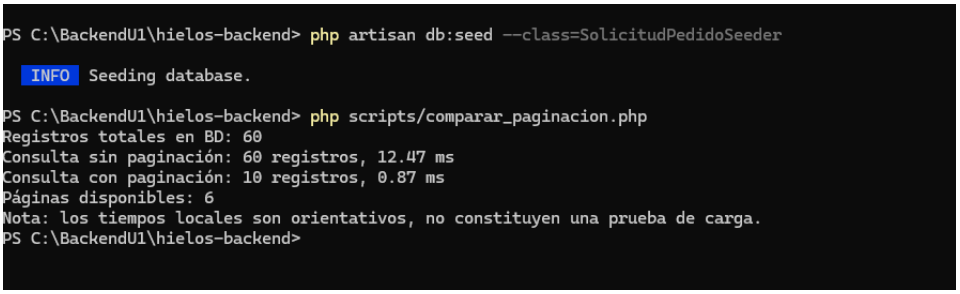


Figura 6. Comparación ejecutada en PowerShell: 60 registros frente a 10 paginados.

También se guardaron capturas de GET, PUT, reinicio, DELETE, validaciones, rendimiento y PHPUnit en la carpeta Evidencias del proyecto.

La colección Postman está en docs/Hielos_del_Maipo_U1.postman_collection.json. No se incluyeron .env ni la base de datos local.

Trabajo desarrollado con datos ficticios y pruebas ejecutadas en un entorno local.